



Section 1:

Sequence to Sequence Model

Sequence-to-Sequence Learning

Welcome to the world of sequence transduction, where systems map input sequences to output sequences.

Crucial characteristic: Input length $\neq$ Output length.

Key Applications:

- Machine Translation (Nepali $\rightarrow$ English)
- Speech Recognition
- Text Summarization

Why Traditional Neural Networks Fail

Traditional architectures like standard CNNs or MLPs are designed for **fixed-size** inputs and outputs.

- **Image Classification:** 1 Image $\rightarrow$ 1 Label (e.g., Cat)
- **Machine Translation:** N words $\rightarrow M$ words

Because vocabulary sizes are massive and output lengths are dynamic, a fixed-size feed-forward network cannot naturally translate sentences.

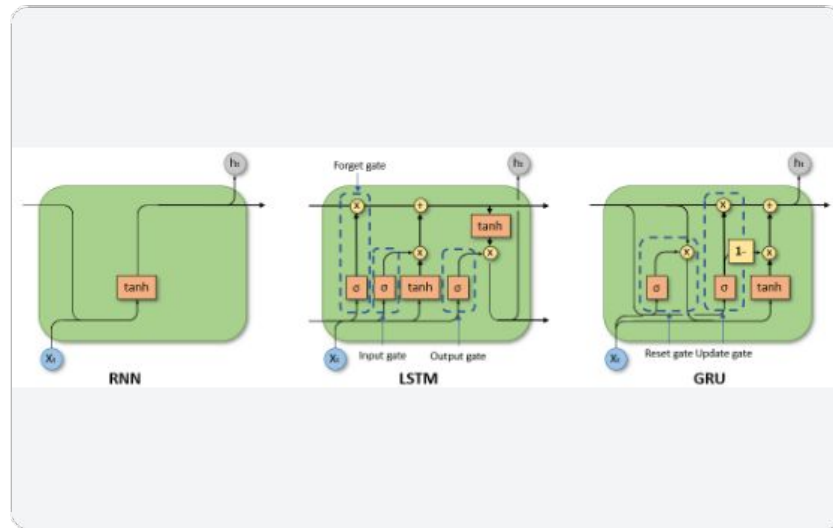
Nepali to English Translation using Sequence to Sequence Modelling

Review: Sequential Information Flow

Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTMs) process data sequentially.

- Input words are processed one by one.
- A **Hidden State** is maintained and updated at each time step.
- The **Final Hidden State** contains information accumulated from the entire input sequence.

म → आज → विद्याल
य → जान्छु → [Final State]



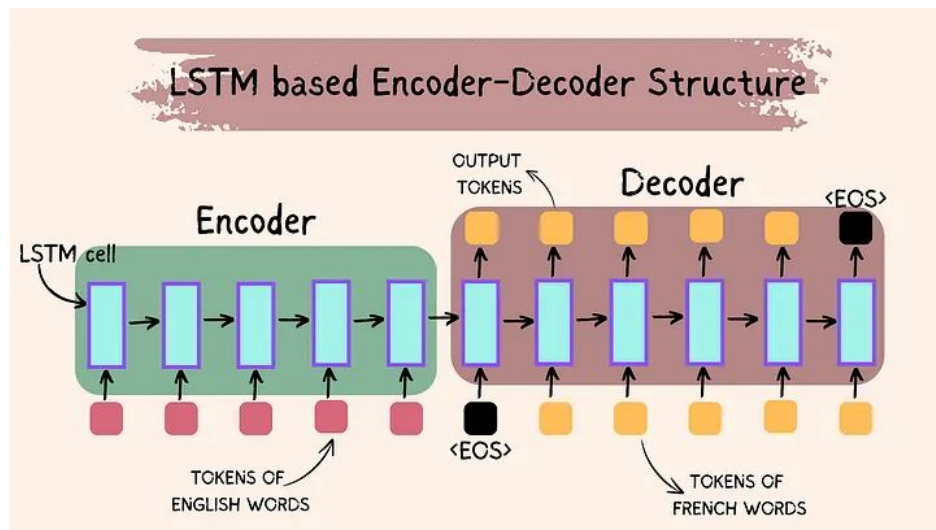
Encoder-Decoder Architecture

The standard Seq2Seq model consists of three main components:

1. **Encoder:** Processes the input sequence.
2. **Context Vector:** The final hidden state of the encoder, summarizing the input.
3. **Decoder:** Generates the output sequence conditionally based on the context vector.

म → आज → विद्याल → जान्छु → [Final State]
य

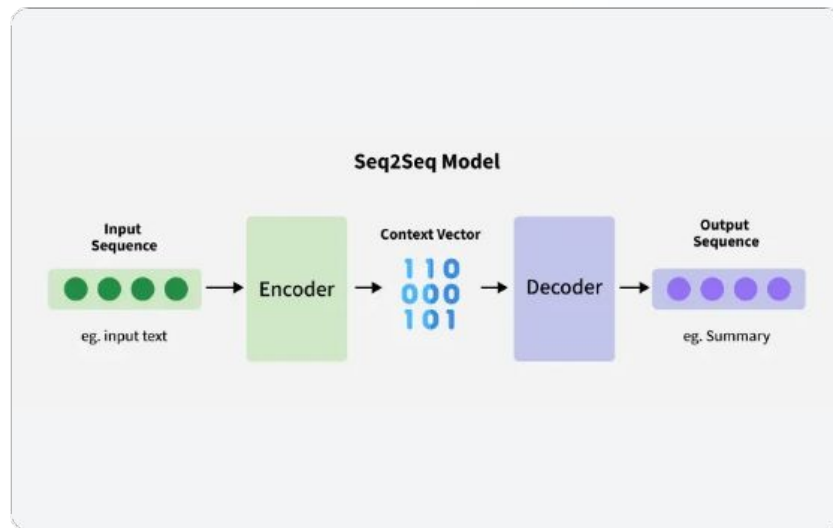
Nepali to English →



The Encoder

The Encoder reads the source sentence "म आज विद्यालय जान्छु" one word at a time.

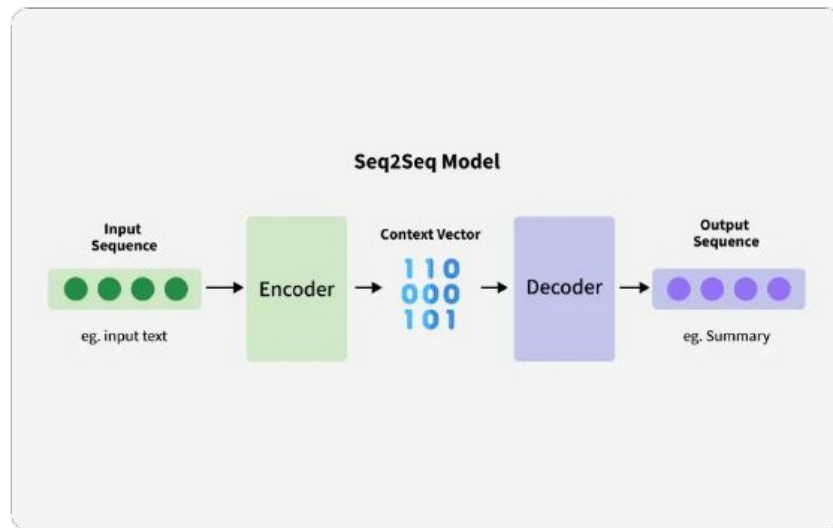
- It produces a sequence of hidden states.
- We typically ignore the intermediate outputs.
- The **final hidden state** becomes the Context Vector, acting as a numeric summary of the Nepali sentence.



The Decoder

The Decoder generates the English sentence "I go to school today" conditionally.

- Initializes its hidden state with the Context Vector.
- Starts generation using a special token.
- Each generated word becomes the input for the *next* time step.
- Stops when it generates the token.



Seq2Seq Mathematics

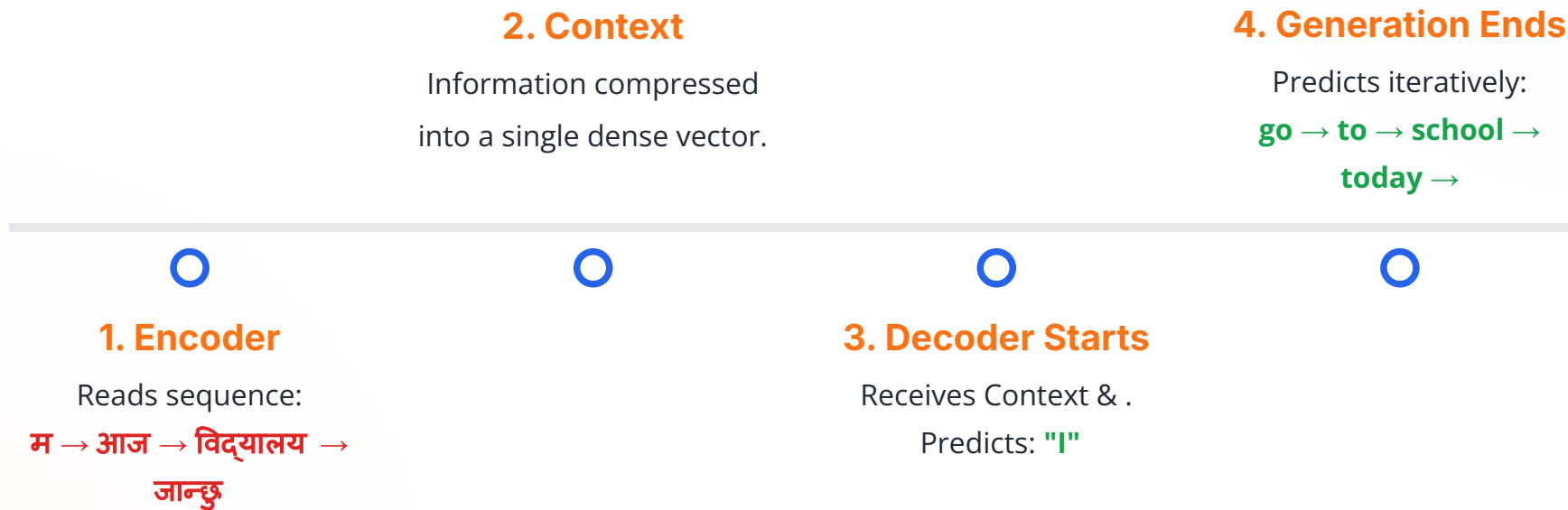
$$\text{Encoder: } h_t = \text{LSTM} (x_t , h_{t-1})$$

$$\text{Context: } c = h_T$$

$$\text{Decoder: } s_t = \text{LSTM} (y_{t-1} , s_{t-1} , c)$$

$$\text{Output: } P (y_t) = \text{softmax} (Ws_t + b)$$

Complete Translation Flow





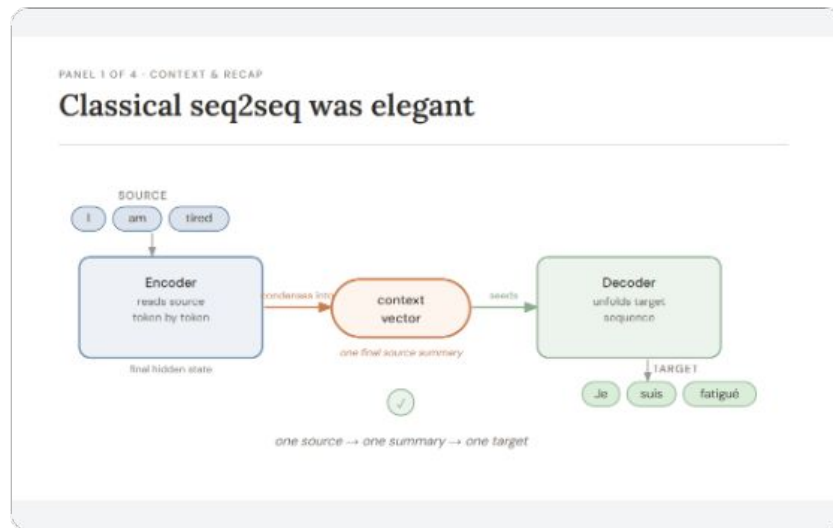
Section 3:

Limitations of Seq2Seq Models and Attention Mechanism

Limitation: The Context Vector Bottleneck

In standard Seq2Seq, the **entire** Nepali sentence must be compressed into a **single fixed-size vector**.

- Works well for short sentences (up to ~20 words).
- Fails drastically on long paragraphs.
- Information is lost due to over-compression.
- The decoder has to generate the entire English translation relying only on this single vector.



The Fading Memory Problem

"The earlier words in the source sentence are slowly forgotten as the LSTM processes more and more words."

When translating a 50-word Nepali sentence, by the time the encoder reaches the 50th word, the information from the 1st word is heavily diluted. This degrades translation quality for long-range dependencies.

Motivation for Attention

What if...

...the decoder could look back at EVERY encoder hidden state during generation?

Instead of relying on one final summary vector, the decoder selectively focuses (attends) to relevant parts of the source sentence for each English word it generates.

Attention Mechanism (Conceptual)



1. Scores

The decoder compares its current state with **all** encoder hidden states to calculate relevance scores.



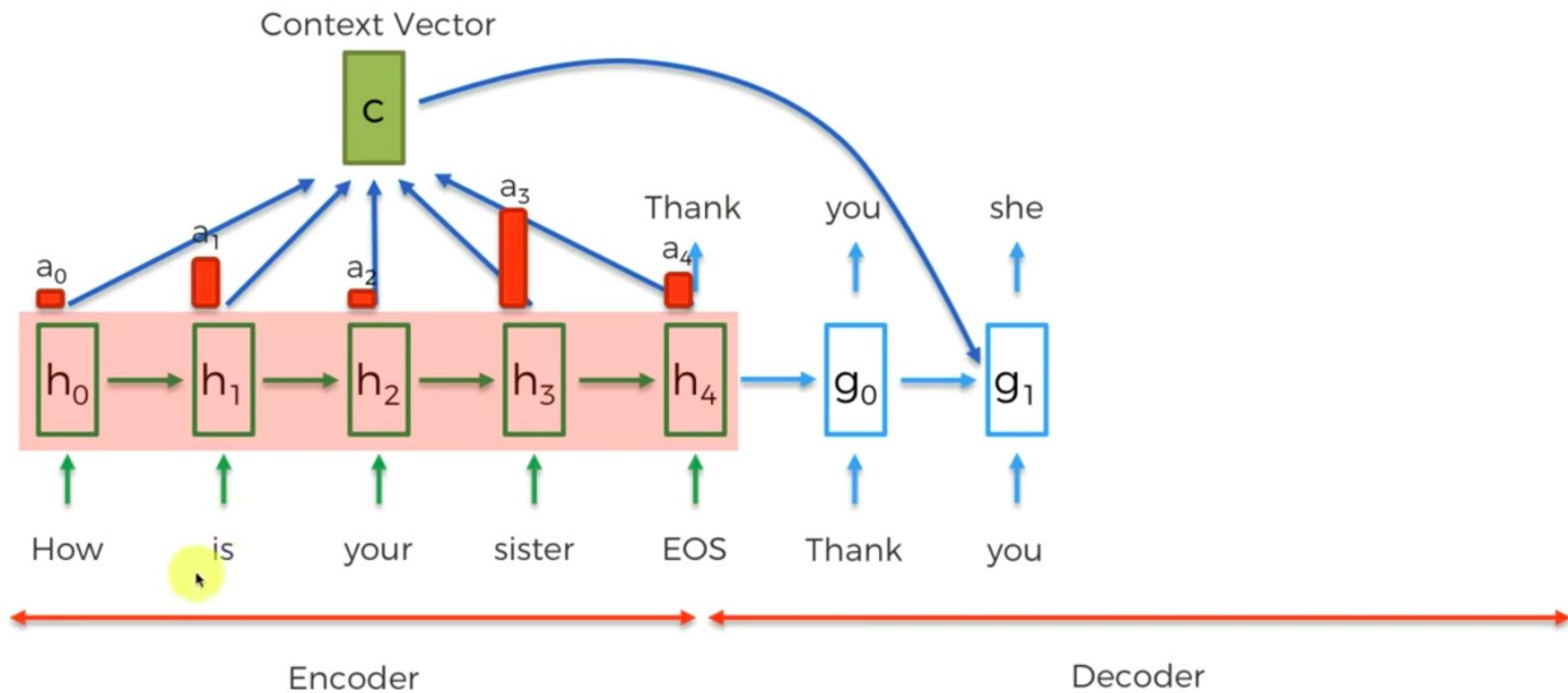
2. Weights

Scores are passed through a Softmax function to create Attention Weights (probabilities summing to 1).



3. Weighted Sum

Encoder states are multiplied by their weights and summed to create a custom, dynamic Context Vector.



Seq2Seq vs. Attention

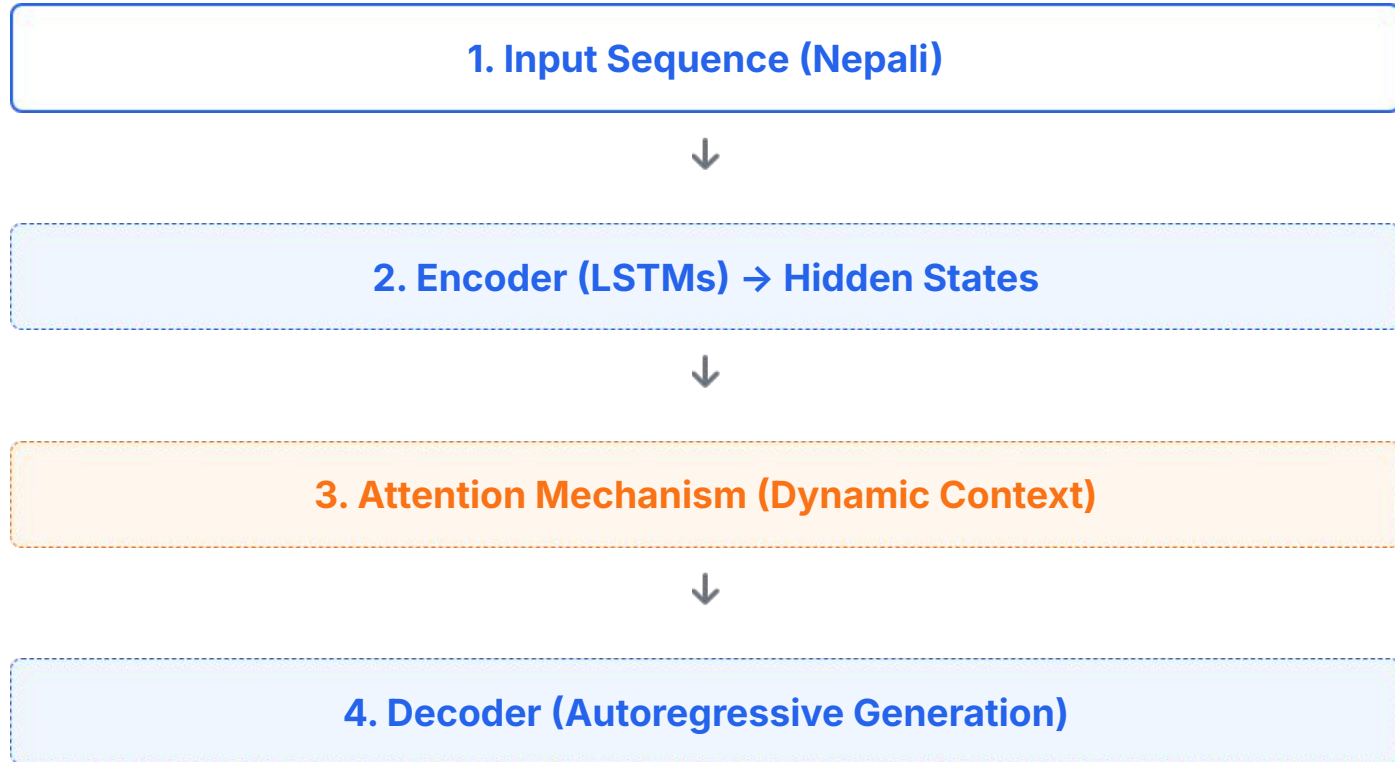
Standard Seq2Seq

- **Context:** Single, static vector.
- **Information Flow:** Bottlenecked at the end of the encoder.
- **Performance:** Degrades rapidly on long sequences.
- **Alignment:** Implicit and opaque.

Seq2Seq with Attention

- **Context:** Dynamic, unique vector per decoding step.
- **Information Flow:** Direct access to all inputs.
- **Performance:** Excellent on long sequences.
- **Alignment:** Highly interpretable (can visualize attention maps).

Summary Flowchart





Section 2: Decoding Strategy

Why Decoding Strategy Matters

The decoder outputs a probability distribution over the entire English vocabulary. We must decide *how* to pick the next word.

Given: I go to ____



Greedy Search

The Strategy

Always select the word with the highest probability at the current time step.

$$\operatorname{argmax} P(y_t \mid y_{1..t-1}, x)$$

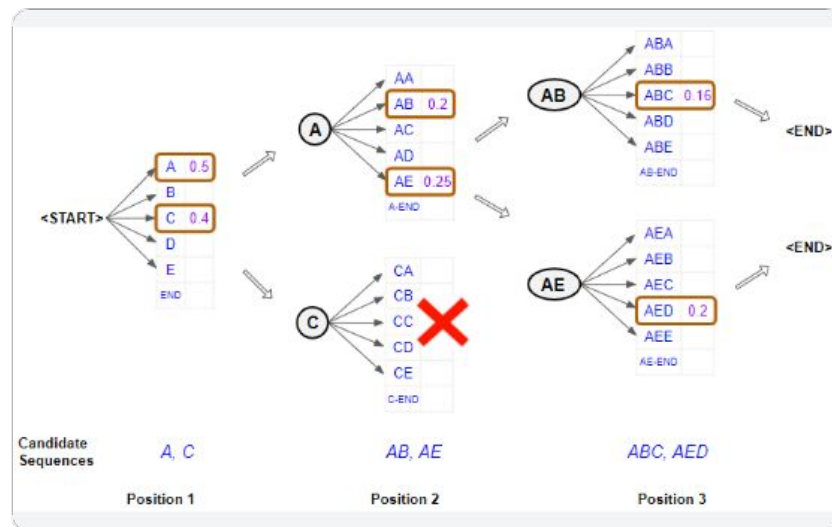
The Trade-off

- **Advantage:** Very fast and computationally cheap.
- **Disadvantage:** Suboptimal. Choosing the local best word might lead to a dead end, ruining the overall sentence probability.

Beam Search

Instead of keeping just one word, keep the top **K** most probable partial translations at every step. (K = *Beam Width*)

- Expands all K candidates.
- Calculates joint probabilities.
- Prunes the tree back down to top K.
- Finds a much better global sequence.



Greedy vs. Beam Search

Criteria	Greedy Search	Beam Search ($K > 1$)
Speed	Very Fast	Slower ($O(K)$)
Memory	Minimal ($O(1)$)	Higher ($O(K)$)
Search Space	Single Path	Multiple Paths
Translation Quality	Often suboptimal / repetitive	Higher quality / grammatically sound

Concept Quiz

- 1. Why is the Context Vector called a bottleneck in standard Seq2Seq?
- 2. Why does Beam Search usually outperform Greedy Search?
- 3. How does the Attention mechanism conceptually solve the Seq2Seq bottleneck?